

```
// =====
// CreatePouchFlow.jsx
// Cowry · "Open a pouch" creation flow
// =====
// 5 screens with internal state – Welcome splash, Pick icon,
// Name it, Goal (optional), Success splash. Calls onComplete
// with the new pouch object when finished.
//
// USAGE:
//   import CreatePouchFlow from './components/CreatePouchFlow';
//
//   const [creatingPouch, setCreatingPouch] = useState(false);
//
//   {creatingPouch && (
//     <CreatePouchFlow
//       onComplete={ (newPouch) => {
//         // newPouch = { name, emoji, category, saved, goal, currency }
//         addPouchToList(newPouch);
//         setCreatingPouch(false);
//       }}
//       onCancel={() => setCreatingPouch(false)}
//     />
//   )}
//
// FILE PLACEMENT:
//   src/components/CreatePouchFlow.jsx
//   src/components/CreatePouchFlow.css
// =====
```

```
import { useState } from 'react';
import PouchIcon from './PouchIcon';
import './CreatePouchFlow.css';
```

```
// — Icon options shown on the picker grid —
```

```
const POUCH_CATEGORIES = [
  { id: 'savings', emoji: '💰', label: 'Savings', suggestions: ['General savi
  { id: 'travel', emoji: '✈️', label: 'Travel', suggestions: ['Trip home',
  { id: 'rainy', emoji: '☔', label: 'Rainy day', suggestions: ['Rainy day fu
  { id: 'gift', emoji: '🎁', label: 'Gift', suggestions: ['Birthday gif
  { id: 'birthday', emoji: '🎂', label: 'Birthday', suggestions: ["Mama's birth
  { id: 'wedding', emoji: '💍', label: 'Wedding', suggestions: ["Sister's wed
  { id: 'school', emoji: '🎓', label: 'School', suggestions: ['School fees'
  { id: 'family', emoji: '👨‍👩‍👧', label: 'Family', suggestions: ['Family medic
  { id: 'home', emoji: '🏠', label: 'Home', suggestions: ['New home', '']
```

```

    { id: 'car',          emoji: '🚗', label: 'Car',          suggestions: ['New car', 'C
    { id: 'faith',       emoji: '🙏', label: 'Faith',        suggestions: ['Tithes', 'Ta
    { id: 'custom',      emoji: '✨', label: 'Custom',       suggestions: [] },
  ];

```

```

export default function CreatePouchFlow({ onComplete, onCancel }) {
  const [step, setStep] = useState(1);
  const [selectedCategory, setSelectedCategory] = useState(null);
  const [pouchName, setPouchName] = useState('');
  const [hasGoal, setHasGoal] = useState(false);
  const [goalAmount, setGoalAmount] = useState('');

  const goNext = () => setStep(step + 1);
  const goBack = () => {
    if (step === 1) {
      onCancel?.();
    } else {
      setStep(step - 1);
    }
  };

  const handleCategoryPick = (cat) => {
    setSelectedCategory(cat);
    // Smart default name = first suggestion if any
    if (cat.suggestions.length > 0 && !pouchName) {
      setPouchName(cat.suggestions[0]);
    }
  };

  const handleFinish = (skipGoal = false) => {
    const goal = skipGoal ? null : parseFloat(goalAmount.replace(/^[^0-9.]/g, ''))
    setStep(5); // Move to success
    setTimeout(() => {
      onComplete?.({
        name: pouchName,
        emoji: selectedCategory.emoji,
        category: selectedCategory.id,
        saved: 0,
        goal,
        currency: 'USD',
      });
    }, 100);
  };

  const handleSuccessDone = () => {
    onComplete?.({
      name: pouchName,

```

```

        emoji: selectedCategory.emoji,
        category: selectedCategory.id,
        saved: 0,
        goal: hasGoal ? (parseFloat(goalAmount.replace(/^[^0-9.]/g, '')) || null) :
        currency: 'USD',
    });
};

// Format goal as currency on blur
const handleGoalBlur = () => {
    const num = parseFloat(goalAmount.replace(/^[^0-9.]/g, ''));
    if (!isNaN(num)) {
        setGoalAmount(num.toLocaleString('en-US', { minimumFractionDigits: 2, maximumFractionDigits: 2 }));
    }
};

// Progress bar percentage
const progressPct = step === 1 ? 5 : step === 2 ? 33 : step === 3 ? 66 : 95;

return (
    <div className="cp-overlay">
        <div className="cp-screen">

            {/* — HEADER (back, progress, step) — hidden on success — */}
            {step < 5 && (
                <div className="cp-header">
                    <button className="cp-back-btn" onClick={goBack} aria-label="Back">
                        {step === 1 ? 'x' : '<'}
                    </button>
                    <div className="cp-progress-line">
                        <div className="cp-progress-fill" style={{ width: `${progressPct}%` }}>
                        </div>
                        <div className="cp-step-num">{step}/4</div>
                    </div>
                </div>
            )}

            {/* — STEP 1: WELCOME SPLASH — */}
            {step === 1 && (
                <div className="cp-body cp-splash">
                    <div className="cp-splash-icon">
                        <PouchIcon size={140} />
                    </div>
                    <h1 className="cp-splash-h">Let's open a <em>pouch.</em></h1>
                    <p className="cp-splash-sub">
                        Set money aside for the moments that matter.
                    </p>
                    <button className="cp-btn-primary" onClick={goNext}>

```

```

        Get started →
      </button>
    </div>
  )}

  {/* — STEP 2: PICK AN ICON — */}
  {step === 2 && (
    <div className="cp-body">
      <h1 className="cp-q-h">What's it <em>for?</em></h1>
      <p className="cp-q-sub">Pick an icon – you can rename it next.</p>

      <div className="cp-icon-grid">
        {POUCH_CATEGORIES.map(cat => (
          <button
            key={cat.id}
            className={`cp-icon-tile ${selectedCategory?.id === cat.id ? 's
            onClick={() => handleCategoryPick(cat)}
          >
            <div className="cp-icon-tile-emoji">{cat.emoji}</div>
            <div className="cp-icon-tile-label">{cat.label}</div>
          </button>
        ))}
      </div>

      <button
        className="cp-btn-primary"
        onClick={goNext}
        disabled={!selectedCategory}
        style={{ opacity: selectedCategory ? 1 : 0.4 }}
      >
        Continue →
      </button>
    </div>
  )}

  {/* — STEP 3: NAME IT — */}
  {step === 3 && (
    <div className="cp-body">
      <h1 className="cp-q-h">Name your <em>pouch.</em></h1>
      <p className="cp-q-sub">Make it personal – for who, for what.</p>

      <div className="cp-selection-row">
        <div className="cp-selection-emoji">{selectedCategory.emoji}</div>
        <div className="cp-selection-label">Your selection</div>
      </div>

      <div className="cp-field">

```

```

    <div className="cp-field-label">Pouch name</div>
    <input
      type="text"
      className="cp-field-input"
      value={pouchName}
      onChange={e => setPouchName(e.target.value)}
      placeholder="What's this pouch for?"
      autoFocus
    />
  </div>

  {selectedCategory.suggestions.length > 0 && (
    <>
      <div className="cp-suggestions-label">SUGGESTIONS</div>
      <div className="cp-suggestions">
        {selectedCategory.suggestions.map((s, i) => (
          <button
            key={i}
            className="cp-suggestion-chip"
            onClick={() => setPouchName(s)}
          >
            {s}
          </button>
        ))}
      </div>
    </>
  )}

  <button
    className="cp-btn-primary cp-btn-bottom"
    onClick={goNext}
    disabled={!pouchName.trim()}
    style={{ opacity: pouchName.trim() ? 1 : 0.4 }}
  >
    Continue →
  </button>
</div>
)}

{/* — STEP 4: SET GOAL (OPTIONAL) — */}
{step === 4 && (
  <div className="cp-body">
    <h1 className="cp-q-h">Set a <em>goal?</em></h1>
    <p className="cp-q-sub">Optional – but goals help you get there faster
    </p>
    <button
      className="cp-toggle-row"

```

```

      onClick={() => setHasGoal(!hasGoal)}
    >
      <div className="cp-toggle-text">
        <div className="cp-toggle-label">Add a goal</div>
        <div className="cp-toggle-sub">Track progress as you save</div>
      </div>
      <div className={`cp-toggle ${hasGoal ? 'on' : ''}`}>
        <div className="cp-toggle-knob" />
      </div>
    </button>

    {hasGoal && (
      <>
        <div className="cp-field">
          <div className="cp-field-label">Goal amount</div>
          <input
            type="text"
            className="cp-field-input"
            value={goalAmount}
            onChange={e => setGoalAmount(e.target.value)}
            onBlur={handleGoalBlur}
            placeholder="$0.00"
            inputMode="decimal"
            autoFocus
          />
        </div>

        <div className="cp-goal-preview">
          <div className="cp-goal-preview-row">
            <div className="cp-goal-preview-emoji">{selectedCategory.emoj
            <div className="cp-goal-preview-name">{pouchName}</div>
          </div>
          <div className="cp-goal-bar">
            <div className="cp-goal-bar-fill" style={{ width: '0%' }} />
          </div>
          <div className="cp-goal-row">
            <span className="cp-goal-saved">$0.00</span>
            <span className="cp-goal-target">
              of {goalAmount ? `${$}${goalAmount}` : '$0.00'}
            </span>
          </div>
        </div>
      </>
    )}

    <button
      className="cp-btn-primary cp-btn-bottom"

```

```

        onClick={() => handleFinish(!hasGoal)}
        disabled={hasGoal && !goalAmount.trim()}
        style={{ opacity: (!hasGoal || goalAmount.trim()) ? 1 : 0.4 }}
    >
        Open my pouch →
    </button>
    {!hasGoal && (
        <button className="cp-btn-skip" onClick={() => handleFinish(true)}>
            Skip – open without a goal
        </button>
    )}
</div>
)}

{/* — STEP 5: SUCCESS SPLASH — */}
{step === 5 && (
    <div className="cp-body cp-splash cp-success">
        {/* Confetti */}
        <svg className="cp-confetti" viewBox="0 0 200 400" preserveAspectRatio="xMidYMid" >
            <circle cx="30" cy="80" r="3" fill="#c9a04a" />
            <circle cx="170" cy="100" r="2" fill="#e4c070" />
            <circle cx="50" cy="140" r="2.5" fill="#c9a04a" />
            <circle cx="180" cy="180" r="2" fill="#e4c070" />
            <circle cx="20" cy="220" r="2" fill="#c9a04a" />
            <circle cx="160" cy="60" r="2.5" fill="#c9a04a" />
            <rect x="140" y="135" width="3" height="6" fill="#e4c070" transform="rotate(45)" />
            <rect x="40" y="180" width="3" height="6" fill="#c9a04a" transform="rotate(-45)" />
            <rect x="170" y="220" width="3" height="6" fill="#e4c070" transform="rotate(45)" />
        </svg>

        <div className="cp-splash-icon cp-splash-icon-success">
            <PouchIcon size={140} stroke="#f5ebd6" accent="#c9a04a" />
        </div>

        <h1 className="cp-splash-h cp-splash-h-success">
            Your pouch <em>is open.</em>
        </h1>

        <p className="cp-splash-sub cp-splash-sub-success">
            {pouchName} is ready for your first save.
        </p>

        <div className="cp-success-card">
            <div className="cp-success-card-row">
                <div className="cp-success-card-emoji">{selectedCategory.emoji}</div>
                <div className="cp-success-card-name">{pouchName}</div>
            </div>
            <div className="cp-success-bar">
                <div className="cp-success-bar-fill" style={{ width: '0%' }} />
            </div>
        </div>
    </div>
)}

```

```

        </div>
        <div className="cp-success-row">
            <span className="cp-success-saved">$0.00</span>
            {hasGoal && goalAmount && (
                <span className="cp-success-target">of ${goalAmount}</span>
            )}
        </div>
    </div>

    <button
        className="cp-btn-primary cp-btn-success"
        onClick={handleSuccessDone}
    >
        Add money to it →
    </button>
    <button
        className="cp-btn-skip cp-btn-skip-success"
        onClick={handleSuccessDone}
    >
        Done — back to home
    </button>
</div>
    )}

</div>
</div>
);
}

```